

LAYER 8 OF 13

SECURITY RLS

Making Sure Nobody Sees Data They Shouldn't

TIER 1 — SOLOPRENEUR

MATT MURPHY .AI

mattmurphy.ai

Part of the Builder Certification Series

What This Kit Covers

What This Kit Covers

This kit covers Layer 8: Security & RLS—protecting your users' data from people who shouldn't see it. When someone signs up for your app, they expect their information stays private. Their profile, their orders, their messages—none of it should be visible to other users. Security is how you keep that promise.

RLS stands for Row-Level Security. Here's what that means in plain English: your database has rows of data—one row per user, one row per order, one row per message. RLS is a set of rules that says "each user can only see their own rows." Without it, a curious person could potentially see everyone else's data. With it, the database itself enforces the walls between users. You don't write RLS rules by hand—you describe what you need to your AI coding tool, and it sets up the rules. Your job is knowing what to ask for and checking that it worked.

This kit also covers the other security basics every app needs: making sure data travels safely over the internet (HTTPS), preventing common attacks that hackers use against web apps, keeping your secret keys and passwords out of your code, and making sure your app only talks to websites it's supposed to. Start at Tier 1. You don't need to become a security expert—you need to know enough to make sure AI builds the right protections.

Why It Matters

A data breach is the fastest way to kill trust in your app. If one user can see another user's private information—even by accident—you have a serious problem. It doesn't matter how good your design is or how fast your app loads. If people's data isn't safe, nothing else matters. And unlike a broken button or a slow page, a security problem can have real legal consequences.

Here's the uncomfortable truth for vibecoders: AI coding tools don't automatically build secure apps. They'll build you a beautiful login screen, but they might skip the part where each user can only see their own data. They'll store your database password right in the code where anyone can find it. They'll leave the front door open unless you specifically tell them to lock it. This kit teaches you exactly what to ask for so your AI builds those locks correctly.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here's a quick look at who each tier is for and what you'll prove you can do.

Tier

Who It's For

What You Prove

Tier 1

Solo builders shipping their first app with user accounts

You can verify that each user only sees their own data, that connections are encrypted, and that common attack vectors are blocked.

Tier 2

Growing apps with teams, roles, and shared data

You can manage role-based access, audit security configurations, and implement defense-in-depth strategies across your application.

Tier 3

Platforms handling sensitive data with compliance requirements

You can govern security at scale—SOC 2 readiness, automated vulnerability scanning, penetration testing, and security incident response.

TIER 1 CERTIFICATION GOAL

You can verify that your app keeps each user's data private, uses encrypted connections, blocks common web attacks, and stores secrets safely—all by directing your AI coding tool with the right prompts.

What You Need to Know

You don't need to become a cybersecurity professional. You need to understand a handful of concepts so you can tell AI what protections to build and verify they're actually in place.

Row-Level Security (RLS): This is the rule system that makes sure User A can't see User B's data. If you're using Supabase (a popular database tool for vibecoders), RLS is built in—but it's turned off by default. That means when you create a new table, every user can see every row unless you turn RLS on and write a policy. Your AI tool can do this, but you have to ask for it explicitly.

HTTPS (encrypted connections): HTTPS means the data traveling between your user's browser and your server is scrambled so nobody in between can read it. Think of it like sending a letter in a locked box instead of on a postcard. Most hosting platforms (Vercel, Netlify, Railway) give you HTTPS automatically, but you need to verify it's actually active—look for the padlock icon in the browser.

Input sanitization (cleaning up what users type): Users type things into your app—names, emails, search terms. A bad actor can type special code into those fields that tricks your app into doing things it shouldn't. Input sanitization means your app cleans up what users type before doing anything with it. Without it, someone could steal data or break your app by typing the right (wrong) thing into a text box.

Secrets management (keeping passwords out of your code): Your app needs passwords and keys to connect to databases, send emails, and use other services. These are called secrets. If you put them directly in your code, anyone who sees your code sees your passwords. Instead, you store them in environment variables—a secure place outside your code that only your server can read. AI tools sometimes put secrets right in the code. Always check.

CORS (who your app talks to): CORS stands for Cross-Origin Resource Sharing. In plain English: it's the list of websites that are allowed to request data from your app. Without proper CORS settings, any website on the internet could try to pull data from your app's backend. You want this locked down to only the domains you control.

Your Toolkit

Your AI coding tool handles the implementation. These are the platforms and tools that make security easier to verify.

Supabase RLS: If you're using Supabase for your database, RLS is the built-in system for data isolation. Ask your AI tool to enable RLS on every table and create policies that restrict each user to their own rows.

Environment variable systems (.env files): Every framework has a way to store secrets outside your code. Your AI tool knows how to use them—but verify that no secrets ended up hardcoded in your actual code files.

Browser DevTools (Network tab): The Network tab in Chrome DevTools lets you see every request your app makes. Check that all connections use HTTPS (not HTTP) and that no sensitive data appears in URLs.

Certification Exam Topics

Every exam question is scenario-based. You'll see a real situation and need to identify what's right, what's wrong, or what to do next.

Data isolation: A user reports they can see another user's profile data. What's the most likely cause, and what do you tell your AI tool to fix?

RLS policies: You created a new database table for customer orders. What's the first security step you need to take before any user accesses it?

HTTPS verification: How do you confirm that your deployed app is using encrypted connections for all data transfers?

Input sanitization: A user types `<script>alert('hacked')</script>` into your app's search bar. What should happen, and what does it mean if the alert actually pops up?

Secrets in code: You're reviewing the code AI generated and find your database password written directly in a file. What's the risk, and what do you ask AI to do instead?

CORS configuration: Your app's API is receiving requests from a website you don't recognize. What setting controls this, and how should it be configured?

Authentication flow: What's the difference between authentication (proving who you are) and authorization (what you're allowed to do), and why does your app need both?

Security headers: Your AI tool built your app but didn't add security headers. What are they, and why should you ask AI to add them?

Common Pitfalls

These are the security mistakes vibecoders make most often. They're easy to make because AI tools don't flag them. If you recognize any of these in your own app, fix them before sitting for the exam.

Forgetting to enable RLS on new database tables. Supabase creates tables with RLS off by default. Every table with user data needs RLS turned on and a policy written—or every user can see every row.

Putting API keys, database passwords, or other secrets directly in your code. AI tools do this all the time. Always check that secrets are in environment variables, never in code files that get uploaded to GitHub or other repositories.

Only checking permissions on the frontend. Hiding a button doesn't mean the action is blocked—anyone can call your API directly. Security must be enforced on the server side, not just hidden in the interface.

Skipping input sanitization because "nobody would do that." Automated bots scan every app on the internet for common vulnerabilities. They don't need a reason to attack your app—they attack everything.

Using HTTP instead of HTTPS, or not checking whether HTTPS is actually active on your deployed app. Without encryption, anyone on the same network can read your users' data in transit.

Never testing security by trying to break it yourself. Log in as one user and try to access another user's data. Try typing weird characters into every input field. If you don't test it, an attacker will.

Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Can you log in as User A and confirm you cannot see User B's data anywhere in the app?

Have you verified that RLS is enabled on every database table that contains user data?

Are all secrets (API keys, database passwords) stored in environment variables, not in your code?

Does your app use HTTPS for every connection, with no HTTP fallbacks?

Have you tested your input fields by typing special characters, script tags, and SQL-like commands to see what happens?

Can you explain the difference between authentication (who you are) and authorization (what you're allowed to do)?

Have you checked your app's CORS settings to make sure only your domains can access your API?

AI Audit Prompt Template

Copy this prompt into your AI coding tool to get a quick security health check on your app. It checks the same things the certification exam covers.

Review my app's security setup and check the following. For each one, tell me pass or fail with a specific example:

RLS policies: Is Row-Level Security enabled on every table with user data? Do the policies correctly restrict each user to their own rows?

Secrets management: Are there any API keys, database passwords, or tokens hardcoded in the source code instead of environment variables?

HTTPS: Are all connections encrypted? Are there any HTTP URLs or mixed-content warnings?

Input sanitization: Are user inputs validated and sanitized before being used in database queries or rendered on pages?

CORS configuration: Is the app's CORS policy restricted to only the domains it should accept requests from?

Authentication and authorization: Does every API endpoint verify the user's identity and check their permissions before returning data?

Security headers: Are headers like Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security present?

Give me an overall score out of 7 and list the top 3 security issues to fix first.

What's Next

Once you've gone through this kit and can answer "yes" to the self-assessment checklist, you're ready for the Layer 8 certification exam at your target tier.

The best way to prepare: take a real app you've built and try to break its security. Log in as one user and try to see another's data. Put weird characters in every text field. Check that your secrets aren't in your code. Every vulnerability you find and fix is proof of work—and exactly what the exam tests.

CERTIFICATION PATHWAY

Associate Builder: Pass all 13 layers at Tier 1

Certified Builder: Pass all 13 layers at Tier 1 + Tier 2

MADE Certified: Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai

© 2026 Matt Murphy .AI. All rights reserved.